

Tutorial 3 Script

Welcome to the tutorial. Today I will demonstrate how to pre-process data you collected with Pandas, and an introduction to regular expressions.

Part 1 Import Libraries

Alright, let's get started with our tutorial. In this section, we'll first import the libraries we need.

Here, we are mainly using Pandas, which is one of the most popular python libraries to analyze and manipulate data.

[run code]

We usually import pandas using its abbreviation pd so that the code later can be shorter.

```
=====
```

Part 2 Data Preprocessing

Next, we move on to data preprocessing. Our goal here is to clean up the dataset so that it's ready for further exploration or modeling.

The first thing we are gonna do is to load our dataset using the read_csv function.

// The file is stored in my Google Drive under the Tutorial3 folder.

By specifying the path of the dataset file, pandas will read the csv file and stores it into a DataFrame, which here we name it as 'data'

[run code]

now this DataFrame contains all the rows and columns from the movie metadata dataset.

```
=====
```

Then, we are using the dot head function to show the first 5 lines of the file. By default, this function shows the top five rows.

[run code]

Each row here represents a movie, and each column represents a feature about the film, such as the director's name, film duration, etc.

```
=====
```

Similarly, we can use the dot tail function to look at the last five rows of the dataset.

[run code]

```
=====
=====
```

Now if we want to take a closer look at a specific column, say, the **duration** column, we can use the dot describe function.

This will give us a quick statistical summary of the data in that column

[run code]

if we ran this line of code, we can see the count, meaning there are just over five thousand movies with valid duration values. The average duration is about 107 minutes, and there are also things like the minimum value, in this case is 7, and the maximum value, which means the longest film being 511 minutes, among other aspects.

```
=====
=====
```

We can also access just one column from the dataset.

In pandas, we can use the square brackets and the column name like this:

[run code]

The output shows the entire color column, which means whether the movie is in color or black-and-white.

selecting columns in this way allows us to quickly zoom in on one aspect of the dataset and inspect it more closely.

```
=====
=====
```

Another useful operation is selecting just the few first rows of a single column, in this case, we set K=10, so this line of code

[run code]

will return the first ten values in the color column

```
=====
=====
```

we can also directly specify a subset of all the columns of the dataset. For example here, we only select the color and director_name columns by passing a list of column names inside another square brackets

[run code]

and corresponding contents will be returned.

```
=====
=====
```

another feature that pandas provides is filtering rows based on conditions. In this example, we're selecting only the movies whose duration is greater than 150 minutes.

[run code]

and all data items that satisfies this condition will be returned

```
=====
=====
```

Next we take a look at how to process NaN (which means Not a Number) data using pandas

```
=====
=====
```

First, we check all NaN data in the DataFrame, using the dot `isna` function.

[run code]

This function creates a table of the same shape as the DataFrame, and each entry is either True or False.

The cell block with a value True represents a NaN value.

```
=====
=====
```

With the knowledge of the existence of NaN values in the dataset, we might want to fill in the missing values with something meaningful.

For example, here we replace missing entries in the country column with the label "cs5481"

[run code]

This is just an example, but in practice we might fill it with something like "unknown"

For the duration column, we take a different approach.

Since the duration values are numeric, we can replace missing values with the average mean duration of all movies.

This way, we don't lose rows of data just because they have missing values, and the values filled in are still reasonable.

```
=====
=====
```

Another option is to drop all the rows that contain NaN values all together. This function is the dot `dropna` function, and it removes any row with at least one missing entry.

[run code]

Note that this function can be aggressive if a lot of rows have missing data, because you might lose a big portion of your dataset.

```
=====
=====
```

To be less aggressive, we can use the `how equals all` parameter.

This means that only rows where all values are missing will be removed, while rows that have at least one valid entry will be kept.

[run code]

```
=====
=====
```

We can also use the `thresh` parameter to set a threshold, which specifies at least how many valid values a row must have for it to be preserved.

For example, here we only keep those rows that have at least 25 non-missing values.

[run code]

This is useful when working with very wide datasets with many columns, because it preserves rows that are mostly complete while dropping the ones that are too sparse.

```
=====
=====
```

So far we've been focusing on dropping rows, we can also drop columns that contain missing values.

If we set `axis equals 1` and `how equals all`, pandas will remove columns where every single value is NaN.

[run code]

Note that when we set the `axis` parameter to 1, it is the columns that would be operated on. In the above examples where we don't specify a value for the `axis` parameter, it's assigned with the default value 0, which means that it is the rows that would be operated on.

If we set `how equals any`, then any column that contains at least one missing value will be removed.

```
=====
=====
```

An important part of processing data is to check the data items that are outside a reasonable range.

We can use the filtering function we described above to achieve this.

For example, when we do not wish to include films released after 2015 for further processes, we run this line of code to check all data items we consider undesirable.

[run code]

Another example is to check whether there is any data item with an imdb score over 10. To do this we run this line of code.

[run code]

```
=====
=====
```

Next we introduce how to check duplicated or replicated rows using pandas. First, we use the dot duplicated function to check whether each row is a duplicate of a previous one.

[run code]

As we can see it returns a series of boolean values, where True means the row is a duplicate.

```
=====
=====
```

As we can see there is no duplicated lines in the movie metadata file, we here use a demo data to show how to process cases where there are duplicated data.

Here is a simple DataFrame with three columns: brand, style and rating. You can see that some rows are repeated for example Yum Yum with style cup and rating 4 appears twice.

[run code]

```
=====
=====
```

Now if we run the dot duplicated function, we can see that there is a True in the returned content, which marks the line being a repeated line of some line before it.

[run code]

By default, the dot duplicated function marks the first occurrence of each duplicated row as False and the later occurrences as True.

```
=====
=====
```

If we set the parameter keep equals last, then the last occurrence of a duplicated row is marked as False and the earlier ones are marked as True

[run code]

```
=====
```

And if we set keep equals False, pandas will mark all duplicates as True.

[run code]

```
=====
```

We can also look for duplicates in just one column by using the subset argument.
For example, here we check only the brand column

[run code]

In this case, any repeated brand name is flagged as a duplicate, regardless of the other columns.

```
=====
```

Next we introduce how to set constraints on the dataset in terms of data types. For example, here I reload the csv file, but this time I tell pandas that the column num_voted_users should be treated as an integer, and the column title_year should be treated as a string.

[run code]

This makes sure that the data is consistent and avoids mistakes during later analyses.

```
=====
```

We can also rename columns to make them more understandable
For instance, I rename title_year into release_date, and movie_facebook_likes into facebook_likes.

[run code]

This doesn't change the data itself, but it makes it easier for humans to read and work with

```
=====
```

Finally, after cleaning up or adjusting the dataset, we often want to save the results for future use.

Here I save the cleaned dataset into a new file, called cleanfile.csv, using UTF-8 encoding.

[run code]

```
=====
=====
=====
=====
```

2.2 Data Integration

Next we look at some data integration operations using pandas. Sometimes it is crucial to unify the diverse sources of information into a consistent format.

Here I first create two simple DataFrames, df1 and df2.

Both of them have a column called key, and each has some data values.

df1 has a distinctive column named data1, and df2 has its distinctive column named data2.

[run code]

[run code]

```
=====
=====
```

When we use pd dot merge df1, df2 function, pandas will look for the common column between the two DataFrames

In this case, the common column is key, so the two DataFrames are merged automatically using that key.

[run code]

Only data items whose keys are presented in both DataFrames remains in the merged DataFrame.

```
=====
=====
```

Of course if we want to be explicit, we can tell pandas exactly which column to merge on, by using the on argument.

[run code]

The result will be the same as the above. Note that even though the column we intend to merge on can be explicit specified by the on argument, it can only be a common column between the two source DataFrames that can be valid. otherwise an error will be raised.

```
=====
=====
=====
```

Now lets look at a case where the two DataFrames don't share any common column name.

In df3 the key column is called 1key, and in df4 it's called 2key.

In this situation, we use left_on and right_on arguments to tell pandas which columns to match.

By default, pandas performs an inner join, which means it only keeps rows with

keys that exist in both DataFrames.

[run code]

From the results we can also see that, distinctive keys from both source DataFrames, specifically when 1key equals c and 2key equals d, are not included in the final merged DataFrame.

Finally, we can also use other types of joins.

For example, if we merge df1 and df2 using how equals outer, pandas keeps all keys from both DataFrames.

If a key only exists in one DataFrame, the missing value from the other DataFrame column will be filled with NaN.

[run code]

This way, no data is lost during merging.

Next, we take a look at how to use pandas to manipulate data in the dataset. Here we introduce two types of data transformations, they are string transformation and number transformation.

Here is an example of transforming all the values in the director_name column into uppercase letters.

[run code]

Similarly, there is also a function that can transform into lowercase letters.

[run code]

Next, we can use the dot strip function to remove the leading and trailing whitespace characters, as well as special characters like the newline character \n.

[run code]

These are all operations on strings. Pandas also allows us to operate on

numeric values.

Here we look at the raw duration column of the dataset. This is the same grammar as we have covered above.

[run code]

```
=====
```

```
=====
```

Next, we transform the unit of measurement by dividing duration by sixty. This would convert movie durations from minutes into hours

[run code]

```
=====
```

```
=====
```

We can also apply a normalization transformation to the duration column.

[run code]

Through this line of code, we rescale the values to fall between zero and one, based on the minimum and maximum values. This is useful when when we want to bring different features onto a similar scale.

```
=====
```

```
=====
```

We can also standardize the duration column. This means we subtract the mean and divide by the standard deviation.

[run code]

The result has a mean of zero and a standard deviation of one. This is useful when we want to compare distributions or when algorithms assume normally distributed features.

```
=====
```

```
=====
```

And finally, we can also perform discretization using the dot qcut function.

[run code]

This divides the continuous duration values into five groups, where each group contains roughly the same number of data points.

Discretization is useful when we want to turn continuous variables into categorical bins for analyses.

```
=====
```

```
=====
```

3 Regular Expressions

Next we introduce regular expressions.

Regular expression provides us with a way to search for specific string patterns according to our intentions.

Here lists the main rules of regex, sort of building blocks for designing any regular expression.

For example, in the meta characters group, a dot means any new character except newline. For example, “he..o” matches both hello and hezzo.

There are also special sequences, such as slash d, which means any digits from 0 to 9; or slash s, which means whitespace.

And the uppercase versions of special sequences do the opposite. For example, slash capital D means non-digit, and slash capital W means non-word, and so on.

There are also sets, for example, square bracket 0 - 9 end square bracket matches any digits, square bracket lowercase a-z capital A-Z end square bracket matches any letter, lowercase or uppercase, etc., etc., etc.

So with sets, we are just creating a little menu of allowed characters.

```
=====
=====
```

And with these specific detailed grammars, we use this to craft regular expressions to express the specific patterns we want to design. And next we will introduce how to use regular expressions with python

```
=====
=====
```

First, the findall function. Suppose I want to extract all the occurrences of a pattern, for example, all instances of the character combination “ai” in lowercase in the sentence.

[run code]

The result is a list. If no matches are found, it just gives an empty list.

[run code]

```
=====
=====
```

In other circumstances, I don’t need all matches, I just want to know if something exists. For example, where is the first whitespace character in the String “The rain in Spain”

[run code]

So as we can see, the search function returns a match object, and we can get the position.

If the pattern doesn't exist, it will return the value None.

[run code]

```
=====
=====
```

We can also cut a string into pieces using the dot split function.
In this example, we split the sentence by spaces.

[run code]

This returns a list of words. And if i only want the first split, I can set the maximum number of splits to one.

[run code]

```
=====
=====
```

Last but not least, I can use the dot sub function to replace text. Suppose I want to replace every whitespace with the number nine.

[run code]

This would achieve that. And if I only want to replace the first two spaces, I can add a count parameter, that way, only part of the string is modified.

[run code]

```
=====
=====
```

Now we dig deeper into the search function and the match object.
A search will return a match object, if you print it directly, something like this will be printed

[run code]

```
=====
=====
```

Moreover, we can use the dot span, dot string, and dot group on the match object. This regular expression looks for any words that starts with an upper case S

[run code]

the dot span returns the start and end positions of the match,

[run code]

the dot string gives the original text

[run code]

and the dot group shows the exact substring that matched, which, in this case, is Spain

```
=====
=====
```

Finally, let's see a more practical example. Suppose I want to extract all email addresses from a sentence.

The regular expression is designed as this and by running this block of code,

[run code]

the regular expression successfully pulls out both email addresses. This shows that we can describe very complex text structures in a single line.

```
=====
=====
```

Next, please do the following practice